

TRI-CORE REALITY ENGINE

Developer Handoff Specification v1.0

Document Date	June 14, 2026
Status	READY FOR DEVELOPMENT
Author	Product Design — Jon
Classification	Internal / Development Use
Product	Tri-Core Reality Engine — Live Monitoring Dashboard
Version	1.0.0-handoff

✓ READY FOR DEVELOPMENT

Table of Contents

Section 1 — Product Overview

1.1 Product Summary

1.2 Design Philosophy

1.3 Target Users

1.4 Core Feature Set

Section 2 — Technology Stack

2.1 Recommended Stack

2.2 No-Backend Architecture

2.3 Browser Requirements

Section 3 — File & Folder Structure

Section 4 — Global State Schema

4.1 Master State Shape (TypeScript)

4.2 External Data Payload Schema

Section 5 — Canvas Architecture

5.1 Canvas Setup

5.2 Animation Loop

5.3 Coordinate System

5.4 Ring Drawing Spec

5.5 Glow Effect Implementation

5.6 Ring Braiding (Z-order Interleaving)

5.7 Particle System

5.8 Drift Distortion Effect

5.9 SparkLine Component

Section 6 — Module Specs

6.1 Monitor Tab

6.2 Configure Tab

6.3 Simulate Tab

6.4 Export Tab

6.5 Docs Tab

[Section 7 — CSS Design Tokens](#)

[Section 8 — NavBar Spec](#)

[Section 9 — Scenarios Full Definition Table](#)

[Section 10 — Responsive Breakpoints](#)

[Section 11 — Deployment Guide](#)

[Section 12 — Known Limitations & Future Roadmap](#)

[Appendices](#)

[Appendix A — Glyph Set](#)

[Appendix B — Color Reference Table](#)

[Appendix C — Quick-Start Checklist for the Developer](#)

Section 1 — Product Overview

1.1 Product Summary

The **Tri-Core Reality Engine** is a real-time AI system state monitoring dashboard. It visualizes three interdependent cognitive and processing layers — **Narrative**, **Cognitive**, and **Render** — as an animated 3D helix engine rendered on an HTML5

Canvas. Alongside the visual engine, the product provides full data input, configuration, simulation, and export capabilities to the system operator.

This document is the authoritative developer handoff specification. A developer picking this up should be able to build the product from scratch using only the information contained herein. Every architectural decision, schema definition, component behavior, and deployment step is specified explicitly.

1.2 Design Philosophy

- **Dark-void aesthetic — clinical meets mythic.** The product must feel like mission control for an AI consciousness, not a typical analytics dashboard. Every visual decision reinforces this identity.
- **Every metric is alive.** Numbers do not merely display — they animate, react, and pulse. Static readouts are not acceptable for primary metrics.
- **The engine canvas is the source of truth visually.** All configuration changes must reflect immediately on the canvas with zero perceivable delay. No "Apply" buttons on the configure screen.
- **Self-healing as a first-class feature.** The system must visually demonstrate drift detection and autonomous correction. The recovery process is a performance — users should feel the tension and resolution.

1.3 Target Users

- **AI system operators** monitoring live model state in real time
- **Developers** building systems using the Tri-Core architecture, using this dashboard to validate their integrations
- **Researchers** studying system coherence, drift behavior, and cross-layer dependencies

1.4 Core Feature Set

The following table defines all features, their assigned module tab, development priority, and developer notes.

Feature	Module Tab	Priority	Notes
Live engine visualization (animated canvas)	Monitor	P1	Core deliverable. RAF-driven canvas. Must not drop below 60fps on target hardware.
Real-time telemetry panel (sparklines, status)	Monitor	P1	Right-column panel. Values lerp to target on each frame.
Data input: Simulated / Manual JSON / Live Feed	Monitor	P1	Three-mode switcher. Live Feed uses fetch() polling. No proxy.
Per-ring configuration (color, speed, labels, glyphs)	Configure	P1	Accordion per ring. All changes immediately affect canvas.
Preset theme system	Configure	P2	4 themes. Merges partial color overrides into config.
Config persistence (localStorage)	Configure	P1	Zustand persist middleware. Config only — not telemetry history.
Drift scenario injection (6 scenarios)	Simulate	P1	All 6 defined in Section 9. Uses setTimeout-based timeline.
Custom scenario builder	Simulate	P2	UI for authoring new ScenarioDef objects. Optional phase.
Recovery timeline log	Simulate	P1	Visual progress bar + phase label + event log entries during scenarios.
PNG export (canvas snapshot)	Export	P1	1×, 1.5×, 2× scale. Uses canvas.toDataURL + anchor click.
JSON config export	Export	P1	Serialized EngineConfig as formatted JSON file download.
Share link (URL hash encoding)	Export	P1	btoa(JSON.stringify(config)) in URL hash. Loaded on mount.

Feature	Module Tab	Priority	Notes
Embed code generation	Export	P2	Generates an iframe snippet pointing to the share link.
Telemetry CSV export	Export	P2	Exports in-memory telemetry history (last 50 readings per metric).
Documentation module	Docs	P2	Static JSX content. Sidebar nav, no CMS.
URL hash config loading on startup	Global	P1	Runs in App.tsx useEffect. See Section 6.4 for implementation.
Responsive layout (< 1000px)	Global	P2	Side panels slide up from bottom on mobile. See Section 10.

Section 2 — Technology Stack

2.1 Recommended Stack

Layer	Technology	Rationale
Framework	React 18 + Vite	Fast HMR, modern JSX, zero config. Vite's ESM-native dev server is essential for iteration speed.
Language	TypeScript	Type-safe state management is critical for complex engine data. All schemas in Section 4 are typed.
Canvas	HTML5 Canvas API (2D context)	Full control over animation loop. No library overhead. Matches the custom aesthetic requirements exactly.
Styling	CSS Modules + CSS custom properties	No Tailwind needed. The dark theme is bespoke and

Layer	Technology	Rationale
		requires precise token control (see Section 7).
State	Zustand	Lightweight, zero boilerplate, excellent for real-time updates. Persist middleware handles localStorage natively.
Persistence	localStorage via Zustand persist middleware	No backend required. Config survives page refresh. Telemetry is not persisted (ephemeral by design).
Charts / Sparklines	Custom canvas mini-charts (no library)	Matches engine aesthetic. Full control over glow, color, and update timing. See SparkLine spec in Section 5.9.
Export / Download	Native File API + canvas.toDataURL	No dependency. Works entirely in browser. See Section 6.4 for full implementation.
URL Encoding	btoa / atob + URLSearchParams	Share links and config loading. No backend or shortlink service required at v1.
Deployment	Vercel / Netlify / GitHub Pages	Static build. Zero server configuration. No environment variables required. See Section 11.

2.2 No-Backend Architecture

This product is intentionally serverless. All state lives in one of three locations:

- **localStorage** — engine config persistence (via Zustand persist middleware)
- **URL hash** — share links with encoded config snapshot
- **In-memory** — telemetry history arrays, runtime animation state, event log entries

The **Live Feed** data input feature uses the browser's native `fetch()` API to poll a user-supplied URL at a configurable interval. No proxy, no CORS proxy server, and no backend relay are required — though the user-supplied endpoint must return correct CORS headers (see Section 12 for the known limitation).

2.3 Browser Requirements

Category	Detail
Minimum Browsers	Chrome 90+ · Edge 90+ · Firefox 88+ · Safari 15+
Required APIs	Canvas 2D API, requestAnimationFrame, localStorage, Clipboard API
Optional APIs	Web Workers (for heavy telemetry computation if needed in a future version)
Unsupported	Internet Explorer (all versions). No polyfills provided.

Section 3 — File & Folder Structure

The complete repository structure is shown below. Inline comments explain each file's purpose. This is the canonical structure — deviation should be documented in the project README.

```
tri-core-engine/ |— public/ | |— favicon.svg # Tri-ring SVG
icon (see Section 8.2) | |— og-image.png # Social share / Open
Graph image | |— src/ | |— main.tsx # App entry point —
mounts React root | |— App.tsx # Root component: tab
router + hash-config loader | |— index.css # Global CSS
vars, reset, custom scrollbars | | |— engine/ # Core
animation engine (canvas layer) | | |— EngineCanvas.tsx # Canvas
element + RAF loop mount point | | |— renderer.ts # Main draw()
dispatcher called each frame | | |— layers/ | | |—
drawParticles.ts # Background particle field | | |—
drawGovernanceRing.ts # Outer containment ring (green) | | |—
drawMirrors.ts # Three mirror node elements | | |—
drawNarrativeRing.ts # Ring 1 — Narrative layer (blue) | | |—
drawCognitiveRing.ts # Ring 2 — Cognitive layer (purple) | | |—
drawRenderRing.ts # Ring 3 — Render layer (red) | | |—
drawStateOrb.ts # Central state orb + coherence pulse | | |—
utils/ | | |— glow.ts # Bloom/glow helper — multi-pass
stroke | | |— ellipse.ts # Tilted ellipse parametric math
helpers | | |— particles.ts # Particle system class (pool +
update + draw) | | |— lerp.ts # Linear interpolation
utility functions | | |— types.ts # RenderContext,
RingConfig, OrbState types | | |— store/ # Global
state (Zustand) | | |— engineStore.ts # Master store: combines all
slices + persist | | |— configSlice.ts # Ring configs, active
theme, label overrides | | |— telemetrySlice.ts # Live metric values
+ 50-entry history arrays | | |— scenarioSlice.ts # Active scenario
```



```

state + phase tracking | | | | | logSlice.ts # Event log entries
(max 50, FIFO) | | | | | modules/ # The 5 product tabs -
one folder each | | | | | monitor/ | | | | | MonitorTab.tsx #
3-column CSS grid layout container | | | | | DataInputPanel.tsx #
Left panel: mode switcher + inputs | | | | | TelemetryPanel.tsx #
Right panel: CoreStatusCards + log | | | | | CoreStatusCard.tsx #
Per-core metric card component | | | | | SparkLine.tsx #
80x24px mini canvas chart component | | | | | EventLog.tsx #
Scrollable timestamped event feed | | | | | configure/ | | | | |
| | | | | ConfigureTab.tsx # Tab layout container | | | | |
RingConfigurator.tsx # Accordion per ring - all controls | | | | |
ThemePresets.tsx # 4 preset theme selector cards | | | | |
OrbConfigurator.tsx # State orb pulse/size controls | | | | |
MiniPreviewCanvas.tsx # Small live preview canvas | | | | |
simulate/ | | | | | SimulateTab.tsx # Tab layout container |
| | | | | ScenarioCard.tsx # Individual scenario display card |
| | | | | ScenarioRunner.ts # Timeline engine (plain class, not React) |
| | | | | RecoveryTimeline.tsx # Visual progress bar + phase label |
| | | | | CustomBuilder.tsx # Custom scenario builder UI (P2) |
| | | | | export/ | | | | | ExportTab.tsx # Tab layout
container | | | | | PngExporter.ts # Canvas to PNG download logic
| | | | | JsonExporter.ts # Config serialization + JSON download
| | | | | ShareLink.tsx # URL hash generation + clipboard copy |
| | | | | CsvExporter.ts # Telemetry history → CSV download |
| | | | | docs/ | | | | | DocsTab.tsx # Tab layout + sidebar
router | | | | | DocsSidebar.tsx # Anchor-linked navigation sidebar
| | | | | sections/ | | | | | Overview.tsx # Product
overview static content | | | | | Architecture.tsx # Architecture
reference static content | | | | | ApiReference.tsx # External
data payload API reference | | | | | Changelog.tsx # Version
history log | | | | | components/ # Shared UI components
(used across tabs) | | | | | NavBar.tsx # Fixed top nav: logo +
tabs + status pill | | | | | TabButton.tsx # Single nav tab button
with active state | | | | | StatusBadge.tsx # ACTIVE / DRIFTING /
ALERT / OFFLINE badge | | | | | SliderControl.tsx # Labeled range input
with live value display | | | | | ColorPicker.tsx # Styled native
input[type=color] wrapper | | | | | Toggle.tsx # Boolean on/off
toggle switch | | | | | Modal.tsx # Generic overlay modal
container | | | | | Tooltip.tsx # Hover tooltip wrapper component
| | | | | ProgressBar.tsx # Animated horizontal progress bar |
| | | | | hooks/ | | | | | useAnimationLoop.ts # RAF setup and teardown with
cleanup | | | | | useTelemetryTick.ts # setInterval for simulated metric
updates | | | | | useShareLink.ts # Encode/decode URL hash config
round-trip | | | | | useEngineResize.ts # ResizeObserver - updates canvas
on resize | | | | | constants/ | | | | | themes.ts # 4
preset theme color objects | | | | | scenarios.ts # 6 drift
scenario definitions (full) | | | | | defaultConfig.ts # Factory
default EngineConfig values | | | | | glyphs.ts # Glyph
character set for narrative ring nodes | | | | | utils/ | | | | |
encode.ts # btoa/atob config serialization helpers | | | | |
download.ts # Anchor-click file download trigger | | | | |
format.ts # Number formatting, ISO timestamp helpers | | | | |
package.json | | | | | vite.config.ts # Base path set here for GitHub
Pages if needed | | | | | tsconfig.json | | | | | README.md # Setup,
run, deploy instructions

```

Section 4 — Global State Schema

4.1 Master State Shape (TypeScript)

The following TypeScript interface definitions constitute the complete state schema for the engine store. These must be implemented exactly as specified — field names, types, and value ranges are consumed by the canvas renderer and all modules.

```
// store/engineStore.ts — Master State Shape
interface RingConfig {
  label: string;
  color: string; // hex e.g. '#3b9eff'
  glowIntensity: number; // 0-100
  rotationSpeed: number; // multiplier 0.5-3.0
  enabled: boolean;
}
interface NarrativeRingConfig extends RingConfig {
  orbitalNodeCount: number; // 3-8
  activeGlyphs: string[]; // e.g. ['Ω', 'Σ', '∞']
  pulseIntervalMs: number; // manuscript pulse frequency in ms
  manuscriptsEnabled: boolean;
}
interface CognitiveRingConfig extends RingConfig {
  driftSensitivity: number; // 0-100 (threshold for drift trigger)
  gridOverlayEnabled: boolean;
  gridOpacity: number; // 0-1
  telemetryLineCount: number; // 2-10
  hudBracketsEnabled: boolean;
}
interface RenderRingConfig extends RingConfig {
  beamCount: number; // 2-6
  beamIntensity: number; // 0-100
  filmStripEnabled: boolean;
  filmStripSegments: number;
  rayConvergenceEnabled: boolean;
}
interface GovernanceRingConfig {
  color: string;
  rotationSpeed: number;
  nodeCount: number; // 4-12
  activeLabels: string[]; // equation fragments to display on nodes
  containmentLinesEnabled: boolean;
}
interface OrbConfig {
  pulseIntervalMs: number;
  innerSwirlSpeed: number;
  radius: number; // 20-60
}
interface EngineConfig {
  engineName: string;
  subtitle: string;
  version: string;
  narrativeRing: NarrativeRingConfig;
  cognitiveRing: CognitiveRingConfig;
  renderRing: RenderRingConfig;
  governanceRing: GovernanceRingConfig;
  orb: OrbConfig;
  globalSpeed: number; // 0.5-3.0 — multiplies all ring speeds
  activeTheme: string; // 'deepVoid' | 'solarFlare' | 'matrixGreen' | 'arcticNeural'
}
interface CoreMetrics {
  value: number; // current reading 0-100
  history: number[]; // last 50 readings — drives sparkline chart
  status: 'ACTIVE' | 'DRIFTING' | 'ALERT' | 'OFFLINE';
  lastUpdated: number; // unix timestamp in ms
}
interface TelemetryState {
  narrative: CoreMetrics & {
    driftIndex: number; // 0-1 (0 = stable, 1 = fully drifted)
    symbolDensity: number; // events per second
  };
  cognitive: CoreMetrics & {
    scanRate: number; // Hz
    anomalies: number; // integer count
    aaisState: 'LOCKED' | 'SCANNING' | 'ALERT';
  };
  render: CoreMetrics & {
    pipeline: 'CLEAR' | 'BUILDING' | 'ERROR';
    rayDepth: number; // 1-16
  };
  orb: {
    coherenceScore: number; // 0-100
    storyStateOk: boolean;
    cogStateOk: boolean;
    renderStateOk: boolean;
    lastValidated: number; // unix timestamp in ms
    validationRate: number; // validations per second
  };
  updateIntervalMs: number; // simulated update cadence
  dataSource: 'simulated' | 'manual' | 'livefeed';
}
interface LogEntry {
  id: string;
  timestamp: number; // unix ms
  message: string;
  level: 'ok' | 'warn' | 'error' | 'info';
}
interface ScenarioState {
  active: boolean;
  scenarioId: string | null;
  phase: string;
  elapsedMs: number;
  durationMs: number;
  timeline: TimelineEvent[];
}
interface TimelineEvent {
  timestampMs: number;
  message: string;
  level:
```

```
'ok' | 'warn' | 'error'; } interface EngineStore {  config: EngineConfig;
telemetry: TelemetryState;  log: LogEntry[];  scenario: ScenarioState;  //
Actions  updateConfig: (partial: DeepPartial<EngineConfig>) => void;
applyTheme: (themeId: string) => void;  resetConfig: () => void;
injectMetrics: (data: ExternalDataPayload) => void;  pushLog: (entry:
Omit<LogEntry, 'id' | 'timestamp'>) => void;  startScenario: (scenarioId:
string) => void;  cancelScenario: () => void;  advanceScenarioPhase:
(phase: string) => void; }
```

4.2 External Data Payload Schema (JSON Input)

This is the shape of data the user pastes in **Manual JSON** mode, or that is received from a **Live Feed** endpoint via fetch(). All fields are optional — only provided fields update the engine state. Missing fields leave existing telemetry values unchanged.

```
// The shape of data accepted by store.injectMetrics() // Used by: Manual
JSON input, Live Feed polling, Scenario phase effects interface
ExternalDataPayload {  narrativeState?: {    coherence: number;          // 0-
100    driftIndex: number;      // 0-1 (0 = fully stable, 1 = fully drifted)
symbolDensity: number; // events per second  };  cognitiveState?:
{    stability: number;        // 0-100    scanRate: number;          // Hz
anomalies: number;           // integer count  };  renderState?: {    fidelity:
number;          // 0-100    pipeline: 'CLEAR' | 'BUILDING' | 'ERROR';
rayDepth: number;           // 1-16  };  stateOrb?: {    coherenceScore:
number;    // 0-100    lastValidated: string;    // ISO 8601 timestamp, or
the literal string "NOW"  }; }
```

Developer Note — Partial Updates:

injectMetrics()

must perform a deep merge, not a replacement. A payload containing only

renderState

must leave

narrativeState

and

cognitiveState

values untouched. Use Zustand's

set()

with a spread merge pattern or Immer if preferred.

Section 5 — Canvas Architecture

5.1 Canvas Setup

- A single `<canvas>` element fills the center column on the Monitor tab, and the full available space in other contexts.
- Canvas CSS dimensions = 100% width and height of its container. The container controls the layout size.
- A `ResizeObserver` watches the container and triggers canvas resize plus re-initialization of any size-dependent values (ring radii, center point, particle spawn bounds).
- **Retina / HiDPI scaling:** `canvas.width = container.clientWidth * devicePixelRatio`. Then call `ctx.scale(dpr, dpr)` immediately after getting the context. All drawing coordinates are specified in CSS pixels — the DPR scaling is transparent to the draw functions.
- The canvas background is cleared to transparent each frame. The CSS background-color on the container element provides the void black color — this allows the canvas to be composited over other elements if needed.

5.2 Animation Loop

The animation loop uses `requestAnimationFrame` for smooth rendering. The loop is initialized inside a React `useEffect` and cleaned up on unmount. Animation state is kept in a plain mutable object — not in React state — to avoid re-renders.

```
// hooks/useAnimationLoop.ts export function useAnimationLoop( canvasRef:
RefObject<HTMLCanvasElement>, engineStore: EngineStore ) { const rafRef =
useRef<number>(0); const startTimeRef = useRef<number>(performance.now());
useEffect(() => { const canvas = canvasRef.current; if (!canvas)
return; const ctx = canvas.getContext('2d')!; const dpr =
```

```

window.devicePixelRatio || 1; // Mutable animation state (NOT in React
state – avoids re-renders)   const animState = initAnimState();
function frame(now: number) {   const elapsed = now -
startTimeRef.current;         const dt = 1; // normalized delta (can be scaled
for speed changes)           // 1. Clear canvas (CSS pixel dimensions, DPR
already applied via scale)     ctx.clearRect(0, 0, canvas.width / dpr,
canvas.height / dpr);         // 2. Update mutable animation angles and
particle positions             updateAngles(animState, engineStore.config, dt);
// 3. Draw all layers – ORDER IS CRITICAL (back to front)
drawParticles(ctx, animState, engineStore);   drawGovernanceRing(ctx,
animState, engineStore);       drawMirrors(ctx, animState, engineStore);
drawNarrativeRing(ctx, animState, engineStore); drawCognitiveRing(ctx,
animState, engineStore);       drawRenderRing(ctx, animState, engineStore);
drawStateOrb(ctx, animState, engineStore);     rafRef.current =
requestAnimationFrame(frame);   } rafRef.current =
requestAnimationFrame(frame);   return () =>
cancelAnimationFrame(rafRef.current); // cleanup on unmount }, []); //
empty deps – store is read via ref pattern, not subscribed }

```

CRITICAL — Performance Boundary:

animState

is a plain mutable JavaScript object, NOT React state. It holds rotation angles, particle positions, and pulse timers — anything that changes on every frame. The Zustand store is strictly

read-only

from the canvas perspective: it provides configuration and telemetry, but

never receives writes from the animation loop

. Violating this separation will cause severe React re-render thrashing and will destroy frame rate.

5.3 Coordinate System

All drawing functions share a common coordinate convention:

- **Origin:** `cx`, `cy` = canvas center in CSS pixels. Passed to every draw function via the render context object.
- **Rings** use tilted ellipse parametric equations to create the 3D helix illusion.

- **Parametric ring point formula:**

```
// Tilted ellipse parametric equation // θ = angle parameter (0 to 2π) //
tilt = ring tilt angle in radians // rx, ry = semi-major and semi-minor axes
x = cx + rx * cos(θ) * cos(tilt) - ry * sin(θ) * sin(tilt) y = cy + rx *
cos(θ) * sin(tilt) + ry * sin(θ) * cos(tilt)
```

5.4 Ring Drawing Specification

The following table defines the precise geometric and visual parameters for each ring. All radius values are expressed as a percentage of `minDim`, where `minDim = Math.min(canvas.width, canvas.height)` in CSS pixels.

Ring	Ellipse rx	Ellipse ry	Tilt Angle	Base Speed (rad/frame)	Default Color	Glow Layers
Narrative	38% minDim	13% minDim	15°	0.0025	#3b9eff	4
Cognitive	15% minDim	42% minDim	80°	0.0020	#b06ef5	4
Render	32% minDim	28% minDim	45°	0.0030	#f06060	4
Governance	46% minDim	40% minDim	10°	0.0008	#38d97a	2

Developer Note:

All base speeds are multiplied by

`engineStore.config.globalSpeed`

and by the individual ring's

`rotationSpeed`

multiplier before being applied each frame. The actual radians advanced per frame

=

`baseSpeed × globalSpeed × ring.rotationSpeed`

5.5 Glow Effect Implementation

The glow (bloom) effect is achieved through multi-pass concentric ellipse strokes with decreasing opacity at increasing spread radii. This technique is compatible with all Canvas 2D contexts and requires no WebGL or external filters.

```
// utils/glow.ts export function drawGlowEllipse( ctx:
CanvasRenderingContext2D, cx: number, cy: number, rx: number, ry: number,
rotation: number, color: string, intensity: number, // 0-100 from
ring config layers: number = 4 ) { const alpha = (intensity / 100) * 0.8;
for (let i = layers; i >= 0; i--) { const spread = i * 3; // 3px spread
per layer ctx.save(); ctx.translate(cx, cy);
ctx.rotate(rotation); ctx.beginPath(); ctx.ellipse(0, 0, rx + spread,
ry + spread, 0, 0, Math.PI * 2); ctx.strokeStyle = color;
ctx.globalAlpha = alpha * (1 - i / (layers + 1)); // fade outer layers
ctx.lineWidth = i === 0 ? 2.5 : 1; // core ring is thicker ctx.stroke();
ctx.restore(); } }
```

5.6 Ring Braiding (Z-order Interleaving)

The DNA braid illusion — where rings visually weave in front of and behind each other — is achieved without any 3D math. The technique uses arc segment Z-ordering:

1. Divide each ring into **8 arc segments**, each spanning $\pi/4$ radians.
2. Assign a Z-order weight to each segment based on its angular position.
Alternate segments are drawn as "front" or "back."
3. Draw all **"back" segments first** (at reduced opacity), then all **"front" segments** on top.
4. This creates the visual crossing effect with no 3D transformation required.

```
// Segment drawing pseudocode (within a draw ring function) const SEGMENTS =
8; // Pass 1: Draw back segments (appears behind other rings) for (let s =
0; s < SEGMENTS; s++) { const startAngle = (s / SEGMENTS) * Math.PI * 2 +
ring.angle; const endAngle = ((s + 1) / SEGMENTS) * Math.PI * 2 +
ring.angle; const isBack = (s % 2 !== 0); if (isBack)
{ ctx.globalAlpha = 0.3; // dimmed = appears behind
drawArcSegment(ctx, ring, startAngle, endAngle); } } // Pass 2: Draw front
segments (appears in front of other rings) for (let s = 0; s < SEGMENTS; s++)
{ const startAngle = (s / SEGMENTS) * Math.PI * 2 + ring.angle; const
endAngle = ((s + 1) / SEGMENTS) * Math.PI * 2 + ring.angle; const isFront =
(s % 2 === 0); if (isFront) { ctx.globalAlpha = 1.0; // full opacity =
```

```
appears in front    drawArcSegment(ctx, ring, startAngle, endAngle);  } }
ctx.globalAlpha = 1; // always restore after drawing
```

5.7 Particle System

A background particle field provides depth and ambient motion. The system uses a fixed-size object pool to avoid garbage collection pressure during the animation loop.

```
// utils/particles.ts interface Particle {  x: number; y: number;  vx:
number; vy: number;  radius: number;  color: string;  opacity: number;
life: number;          // 0-1, decrements each frame  decay: number;    //
rate of life loss per frame } class ParticleSystem {  private pool:
Particle[] = [];  private active: Particle[] = [];  constructor(private
count: number) {    this.pool = Array.from({ length: count }, () =>
this.createParticle());    this.active = [...this.pool]; // all particles
start active  }  private createParticle(): Particle {    return
{      x: 0, y: 0, vx: 0, vy: 0,      radius: Math.random() * 1.5 + 0.5,
color: '#3b9eff',      opacity: 0,      life: Math.random(),      decay:
Math.random() * 0.003 + 0.001,    };  }  private reset(p: Particle, cx:
number, cy: number, w: number, h: number) {    p.x = cx + (Math.random() -
0.5) * w * 0.9;    p.y = cy + (Math.random() - 0.5) * h * 0.9;    p.vx =
(Math.random() - 0.5) * 0.3;    p.vy = (Math.random() - 0.5) * 0.3;
p.life = 1;    p.decay = Math.random() * 0.003 + 0.001;  }  update(dt:
number, cx: number, cy: number, w: number, h: number) {    for (const p of
this.active) {      p.x += p.vx * dt;      p.y += p.vy * dt;      p.life -=
p.decay * dt;      p.opacity = p.life * 0.4;      if (p.life <= 0)
this.reset(p, cx, cy, w, h);    }    draw(ctx: CanvasRenderingContext2D)
{      for (const p of this.active) {        ctx.globalAlpha = p.opacity;
ctx.fillStyle = p.color;        ctx.beginPath();        ctx.arc(p.x, p.y,
p.radius, 0, Math.PI * 2);        ctx.fill();      }      ctx.globalAlpha =
1; // restore after drawing all particles    } }
```

5.8 Drift Distortion Effect (Cognitive Ring)

When the Cognitive core's stability metric drops below the configured drift sensitivity threshold, the ring's `ry` value is perturbed by a sinusoidal wave. The wave amplitude scales with severity of the drift.

```
// Applied inside drawCognitiveRing.ts on each frame function
getDriftRy(baseRy: number, stability: number, time: number): number {  if
(stability > 70) return baseRy; // no drift above threshold  const
driftAmount = (1 - stability / 70); // 0 = stable, 1 = maximum drift  const
wave = Math.sin(time * 0.05) * driftAmount * baseRy * 0.35;  return baseRy +
wave; } // Usage in drawCognitiveRing: const stability =
engineStore.telemetry.cognitive.value; const ry =
getDriftRy(BASE_COGNITIVE_RY, stability, animState.time); // Use 'ry' instead
of BASE_COGNITIVE_RY for all ring drawing calls this frame
```

Note:

The

time

parameter passed to

getDriftRy

is

animState.time

— an integer incrementing by 1 each frame. This provides a consistent, frame-rate-independent wave regardless of actual elapsed wall time. At 60fps,

time * 0.05

cycles approximately once every 2 seconds.

5.9 SparkLine Component (Mini Canvas)

SparkLine is a small canvas element (80×24 CSS pixels) that renders the last N telemetry readings as a line chart. It re-draws whenever its values prop changes.

```
// modules/monitor/SparkLine.tsx export function SparkLine({ values, color }:  
{ values: number[]; color: string }) { const canvasRef =  
useRef<HTMLCanvasElement>(null); useEffect(() => { const canvas =  
canvasRef.current!; const ctx = canvas.getContext('2d')!; const w =  
canvas.width; const h = canvas.height; ctx.clearRect(0, 0, w, h);  
if (values.length < 2) return; const min = Math.min(...values);  
const max = Math.max(...values); const range = max - min || 1; // avoid  
division by zero ctx.beginPath(); ctx.strokeStyle = color;  
ctx.lineWidth = 1.5; values.forEach((v, i) => { const x = (i /  
(values.length - 1)) * w; const y = h - ((v - min) / range) * (h - 4) -  
2; // 2px padding top/bottom i === 0 ? ctx.moveTo(x, y) : ctx.lineTo(x,  
y); }); ctx.stroke(); }, [values, color]); return <canvas  
ref={canvasRef} width={80} height={24} style={{ display: 'block' }} />; }
```

Section 6 — Module Specs

6.1 Monitor Tab

Layout

The Monitor tab uses a three-column CSS Grid layout that fills the viewport below the navbar. The center column expands to fill all remaining space and contains the engine canvas.

```
/* modules/monitor/MonitorTab.module.css */ .monitor-layout { display:
grid; grid-template-columns: 280px 1fr 280px; gap: 12px; height:
calc(100vh - 56px); /* full height minus navbar */ padding: 12px;
overflow: hidden; /* panels scroll internally, not the layout */
background-color: var(--bg-void); } .left-panel { overflow-y:
auto; } .center-panel { position: relative; } /* canvas fills this */ .right-
panel { overflow-y: auto; }
```

DataInputPanel Component

Props: None — reads from and writes to store directly.

Internal state (React useState):

- `mode: 'simulated' | 'manual' | 'livefeed'`
- `jsonText: string` — textarea value
- `jsonError: string | null` — validation error message
- `liveUrl: string` — user-entered endpoint URL
- `pollInterval: number` — ms between fetches, default 2000
- `pollTimerId: ReturnType<typeof setInterval> | null`

Behavior specification:

- **Mode switch:** clears `jsonError`, stops any active poll timer, resets UI state for the new mode.
- **INJECT DATA button (manual mode):** calls `JSON.parse(jsonText)` inside a try/catch. On success: calls `store.injectMetrics(parsed)`, clears error, shows brief success state on button. On failure: sets `jsonError` with the parse error message.
- **CONNECT button (live feed mode):** calls `setInterval(fetchAndInject, pollInterval)`. Each tick: `fetch(liveUrl) → .json() → store.injectMetrics()`.

If fetch or parse fails: display error in panel, do not crash, do not stop the poll (user must manually disconnect).

- **DISCONNECT button:** calls `clearInterval(pollTimerId)`, resets `pollTimerId` to null.
- **Simulated mode sliders:** directly write to `store.updateSimulatedConfig()` on every `onChange` event — changes apply immediately with no confirm step.

TelemetryPanel Component

- Three `CoreStatusCard` components stacked (Narrative → Cognitive → Render)
- `StateOrbCard` below the three core cards
- `EventLog` at bottom with `max-height: 180px` and `overflow-y: scroll`
- All values read from Zustand store via selector hooks
- Display values lerp toward target on each animation frame using a `useRef` holding current display values — prevents jarring number jumps

CoreStatusCard Component

Props: `core: 'narrative' | 'cognitive' | 'render'`

Renders:

- Label text + colored indicator dot (color matches ring color token)
- `StatusBadge` component showing current `status` field — color-coded per status
- Primary metric value (large mono text) + `SparkLine` chart side by side
- Secondary metrics as key/value label rows below
- `ProgressBar` component for the primary metric (0-100)

EventLog Component

- Reads `store.log` array (maximum 50 entries, FIFO)
- New entries appear at the **top** with a brief CSS fade-in transition

- Level color mapping: `ok` → `#38d97a` · `warn` → `#f59e0b` · `error` → `#ef4444` · `info` → `#5a6a82`
- Entry format: `[HH:MM:SS]` `<icon>` `message` — timestamp is local time
- Auto-scrolls to top (`scrollTop = 0`) whenever a new entry is pushed

6.2 Configure Tab

RingConfigurator Component

Props: `ring: 'narrative' | 'cognitive' | 'render' | 'governance'`

Rendered as an accordion section — collapsed by default, expanded on header click. All inputs are fully controlled components: they read from `store.config[ringKey]` and call `store.updateConfig()` on every change event. Changes propagate to the canvas on the next animation frame (typically <16ms).

- **Color picker:** Native `<input type="color">` styled to match the dark theme. Wrapper component in `components/ColorPicker.tsx`.
- **Sliders:** `SliderControl` component with label, min, max, step, and live numeric readout.
- **Toggles:** `Toggle` component for boolean fields (`enabled`, `gridOverlayEnabled`, `filmStripEnabled`, etc.).
- **Glyph selector** (Narrative ring only): Multi-select list of glyph characters from `constants/glyphs.ts`. Selected glyphs appear on orbital nodes.

ThemePresets Component

Displays 4 preset cards in a 2×2 grid. Each card contains:

- A mini color swatch strip — three colored blocks (Narrative / Cognitive / Render colors)
- Theme name + brief description line
- Click handler: `store.applyTheme(themeId)` — merges the theme color object into the existing config (does not touch non-color fields)

- Active theme card is visually highlighted with a border

```
// constants/themes.ts export const THEMES = { deepVoid:
{ narrativeRing: { color: '#3b9eff' }, cognitiveRing: { color:
'#b06ef5' }, renderRing: { color: '#f06060' }, governanceRing:
{ color: '#38d97a' }, }, solarFlare: { narrativeRing: { color:
'#fb923c' }, cognitiveRing: { color: '#fbbf24' }, renderRing:
{ color: '#f9fafb' }, governanceRing: { color: '#fcd34d' }, },
matrixGreen: { narrativeRing: { color: '#4ade80' }, cognitiveRing:
{ color: '#86efac' }, renderRing: { color: '#dcfce7' },
governanceRing: { color: '#22c55e' }, }, arcticNeural:
{ narrativeRing: { color: '#67e8f9' }, cognitiveRing: { color:
'#a5f3fc' }, renderRing: { color: '#e0f2fe' }, governanceRing:
{ color: '#38bdf8' }, }, } as const;
```

Config Persistence

Uses Zustand's persist middleware with the localStorage adapter. Only the config slice is persisted — telemetry, log, and scenario state are always ephemeral.

```
// store/engineStore.ts – persist setup export const useEngineStore =
create<EngineStore>()( persist( (set, get) =>
({ ...configSlice(set, get), ...telemetrySlice(set,
get), ...logSlice(set, get), ...scenarioSlice(set, get), }),
{ name: 'tri-core-engine-config', partialize: (state) =>
({ config: state.config }), // telemetry excluded } ) );
```

6.3 Simulate Tab

Scenario Definitions Type (constants/scenarios.ts)

```
export interface ScenarioDef { id: string; title: string; description:
string; severity: 'medium' | 'high' | 'critical' | 'test'; durationMs:
number; phases: ScenarioPhase[]; } export interface ScenarioPhase
{ label: string; startMs: number; storeEffects:
Partial<ExternalDataPayload>; // injected into store at phase start
logMessage: string; logLevel: 'ok' | 'warn' | 'error' | 'info'; }
```

Example Scenario — NARRATIVE DRIFT

```
{ id: 'narrativeDrift', title: 'NARRATIVE DRIFT', description:
'Coherence degrades in the Narrative ring. Mirror A fractures. '
+ 'Cognitive ring auto-compensates and drives recovery.', severity:
'medium', durationMs: 8000, phases: [ { label: 'PHASE 1:
COHERENCE DEGRADING', startMs: 0, storeEffects: { narrativeState:
{ coherence: 41, driftIndex: 0.6 } }, logMessage: '⚠ Narrative
coherence drop initiated', logLevel: 'warn', }, { label:
'PHASE 2: MIRROR A FRACTURE', startMs: 2000, storeEffects: {},
logMessage: '⚠ Mirror A: fracture detected', logLevel: 'error', },
{ label: 'PHASE 3: COGNITIVE COMPENSATING', startMs: 4000,
storeEffects: { cognitiveState: { stability: 99 } }, logMessage: '▢
Cognitive ring initiating correction', logLevel: 'info', }, {
label: 'PHASE 4: RECOVERY', startMs: 6500, storeEffects:
{ narrativeState: { coherence: 96, driftIndex: 0.02 } }, logMessage: '✓
Narrative coherence restored', logLevel: 'ok', },
```

```
{      label: 'SYSTEM STABLE',      startMs: 7500,      storeEffects: {},
logMessage: '✓ System stable – all metrics nominal',      logLevel: 'ok',
},    ], }
```

ScenarioRunner Class (modules/simulate/ScenarioRunner.ts)

ScenarioRunner is a plain TypeScript class — not a React component. It owns the setTimeout timeline and coordinates state injection.

```
// modules/simulate/ScenarioRunner.ts import { DEFAULT_TELEMETRY_VALUES }
from '../constants/defaultConfig'; class ScenarioRunner { private
timeouts: ReturnType<typeof setTimeout>[] = []; start(scenario:
ScenarioDef, store: EngineStore) { this.timeouts = []; // clear any
leftover refs store.startScenario(scenario.id); for (const phase of
scenario.phases) { const t = setTimeout(() => { if (!
store.scenario.active) return; // guard: was cancelled mid-run
store.injectMetrics(phase.storeEffects);
store.advanceScenarioPhase(phase.label); store.pushLog({ message:
phase.logMessage, level: phase.logLevel }); }, phase.startMs);
this.timeouts.push(t); } // Auto-complete after full duration
const done = setTimeout(() => { store.cancelScenario();
store.pushLog({ message: '✓ Scenario complete', level: 'ok' }); },
scenario.durationMs); this.timeouts.push(done); } cancel(store:
EngineStore) { this.timeouts.forEach(clearTimeout); this.timeouts =
[]; store.cancelScenario();
store.injectMetrics(DEFAULT_TELEMETRY_VALUES); // restore nominal values
store.pushLog({ message: '🔄 Scenario cancelled – state reset', level: 'info'
}); } } // Export singleton – one runner instance shared across the app
export const scenarioRunner = new ScenarioRunner();
```

6.4 Export Tab

PNG Export

```
// modules/export/PngExporter.ts export function exportCanvasToPng( canvas:
HTMLCanvasElement, scale: 1 | 1.5 | 2, // 1 = native, 1.5 =
1440p quality, 2 = 4K filename = 'tri-core-engine.png' ) { // Create an
offscreen canvas at the desired export resolution const exportCanvas =
document.createElement('canvas'); exportCanvas.width = canvas.width *
scale; exportCanvas.height = canvas.height * scale; const ctx =
exportCanvas.getContext('2d')!; ctx.scale(scale, scale);
ctx.drawImage(canvas, 0, 0); // blit current frame to export canvas const
url = exportCanvas.toDataURL('image/png'); triggerDownload(url,
filename); } // utils/download.ts export function triggerDownload(url:
string, filename: string) { const a = document.createElement('a'); a.href
= url; a.download = filename; document.body.appendChild(a); a.click();
document.body.removeChild(a); }
```

JSON Config Export

```
// modules/export/JsonExporter.ts export function exportConfig(config:
EngineConfig) { const json = JSON.stringify(config, null, 2); const blob
= new Blob([json], { type: 'application/json' }); const url =
URL.createObjectURL(blob); triggerDownload(url, 'tri-core-config.json');
URL.revokeObjectURL(url); // clean up }
```

Share Link System

```
// hooks/useShareLink.ts export function useShareLink() {  const config =
useEngineStore((s) => s.config);  const generateLink = (): string =>
{    const encoded = btoa(JSON.stringify(config));    return `${
>window.location.origin}${window.location.pathname}#config=${encoded}`;  };
const loadFromHash = (): EngineConfig | null => {    const hash =
>window.location.hash;    if (!hash.startsWith('#config=')) return null;
try {      const raw = hash.replace('#config=', '');      return
JSON.parse(atob(raw)) as EngineConfig;    } catch
{      console.warn('[useShareLink] Failed to decode config from URL hash');
return null;    }  };  return { generateLink, loadFromHash }; } //
App.tsx – load shared config on mount useEffect(() => {  const
{ loadFromHash } = useShareLink();  const sharedConfig = loadFromHash();
if (sharedConfig) {    store.updateConfig(sharedConfig);    showBanner('✓
Shared configuration loaded from URL');    // Clear hash so bookmarking the
page doesn't always reload the shared config
>window.history.replaceState(null, '', window.location.pathname);  } },
[]); // runs once on mount
```

CSV Telemetry Export Format

Telemetry history is exported as a flat CSV. Each row represents one historical reading. The history arrays hold the last 50 readings per metric.

```
// CSV column headers and sample rows timestamp,core,metric,value,status
2026-06-14T13:24:01Z,narrative,coherence,97.4,ACTIVE 2026-06-
14T13:24:01Z,cognitive,stability,99.1,ACTIVE 2026-06-
14T13:24:01Z,render,fidelity,94.7,ACTIVE 2026-06-
14T13:24:03Z,narrative,coherence,95.2,DRIFTING 2026-06-
14T13:24:03Z,cognitive,stability,98.8,ACTIVE ...
```

6.5 Docs Tab

The Docs tab is a static content module with no dynamic data requirements. It serves as in-app documentation for system operators and developers integrating with the Tri-Core architecture.

- **Layout:** Sidebar navigation on the left (fixed, anchor links) + scrollable content area on the right.
- **Content:** Hardcoded JSX in the `sections/` files. No CMS, no markdown parser, no external data fetch.
- **Sections:** Overview · Architecture · API Reference · Changelog
- **Implementation note:** All section components are lazy-loaded via `React.lazy()` + `Suspense` to avoid bundling the full docs content into the initial chunk.

Section 7 — CSS Design Tokens

All visual values are defined as CSS custom properties in `index.css`. Components reference these tokens — never hardcode color or spacing values in component CSS files. This enables the theme system to work correctly and ensures visual consistency.

```
/* src/index.css — Global CSS Custom Properties */ :root { /* —
Background Colors _____ */ --bg-void:
#020810; /* Page background — near-black deep space */ --bg-surface:
#080f1e; /* Cards, panels */ --bg-elevated: #0d1728; /* Dropdowns,
modals, tooltips */ /* — Border Colors
_____ */ --border-subtle: rgba(100, 160,
255, 0.12); /* Default card borders */ --border-active: rgba(100, 160,
255, 0.30); /* Focused inputs, hover state */ /* — Core Ring Colors
_____ */ --color-narrative: #3b9eff; /* Ring
1 — Narrative layer */ --color-cognitive: #b06ef5; /* Ring 2 — Cognitive
layer */ --color-render: #f06060; /* Ring 3 — Render layer */ --
color-governance: #38d97a; /* Governance containment ring */ /* —
Status / Alert Colors _____ */ --color-warn:
#f59e0b; /* Warning / degraded state */ --color-error: #ef4444; /*
Critical alert / error events */ /* — Text Colors
_____ */ --text-primary: #e8edf5; /* Main
readable body text */ --text-muted: #5a6a82; /* Secondary labels,
timestamps */ --text-dim: #2a3a52; /* Placeholder text, disabled
state */ /* — Typography _____ */ --
font-mono: 'Courier New', 'Courier', monospace; /* Metric values, code, IDs
*/ --font-body: system-ui, -apple-system, sans-serif; /* UI labels,
descriptions */ --font-size-xs: 11px; --font-size-sm: 12px; --font-
size-base: 14px; --font-size-lg: 16px; /* — Spacing Scale
_____ */ --space-xs: 4px; --space-sm: 8px;
--space-md: 12px; --space-lg: 16px; --space-xl: 24px; /* — Layout
Constants _____ */ --nav-height:
56px; /* Fixed navbar height */ --panel-width: 280px; /* Left and
right panel width */ --border-radius: 10px; /* Standard card radius */
--border-radius-sm: 6px; /* Compact element radius */ /* — Transitions
_____ */ --transition-fast: 150ms
ease; /* Hover states, badge changes */ --transition-med: 300ms
ease; /* Panel slide, mode switches */ } /* — CSS Reset
_____ */ *, *::before, *::after { box-
sizing: border-box; } body { margin: 0; padding: 0; background-color: var(--
bg-void); } button { cursor: pointer; font-family: inherit; } input,
textarea, select { font-family: inherit; } /* — Custom Scrollbar
_____ */ ::-webkit-scrollbar { width: 6px;
height: 6px; } ::-webkit-scrollbar-track { background: var(--bg-void); } ::-
webkit-scrollbar-thumb { background: var(--border-active); }
```

Section 8 — NavBar Specification

8.1 NavBar Component Layout

The NavBar is a fixed-position top bar, height: 56px, spanning full viewport width. It is divided into three zones:

Zone	Position	Contents
LEFT	Left-aligned	Tri-ring SVG logo + product name text + version badge (v1.0)
CENTER	Centered	5 TabButton components: MONITOR · CONFIGURE · SIMULATE · EXPORT · DOCS
RIGHT	Right-aligned	System Status pill + live clock (HH:MM:SS) + Share button

8.2 Tri-Ring SVG Logo

The logo consists of three animated intersecting ellipses, each representing one core ring. Animations are CSS keyframe-driven (no JavaScript).

```
<!-- components/NavBar.tsx — Logo SVG --> <svg width="28" height="28"
viewBox="0 0 28 28" aria-label="Tri-Core Engine">  <!-- Narrative ring –
blue, horizontal orientation -->  <ellipse cx="14" cy="14" rx="11" ry="5"
fill="none"          stroke="#3b9eff" stroke-width="1.5" opacity="0.9"
style="animation: navSpin1 4s linear infinite" />  <!-- Cognitive ring –
purple, vertical orientation, reversed -->  <ellipse cx="14" cy="14" rx="5"
ry="11" fill="none"          stroke="#b06ef5" stroke-width="1.5"
opacity="0.9"          style="animation: navSpin2 5s linear infinite
reverse" />  <!-- Render ring – red, diagonal orientation -->  <ellipse
cx="14" cy="14" rx="9" ry="8" fill="none"          stroke="#f06060" stroke-
width="1.5" opacity="0.9"          transform="rotate(45 14 14)"
style="animation: navSpin3 6s linear infinite" /> </svg> <!-- CSS keyframes
(in index.css or NavBar.module.css) --> <style>  @keyframes navSpin1 { from
{ transform: rotateX(0deg); } to { transform: rotateX(360deg); } }
@keyframes navSpin2 { from { transform: rotateY(0deg); } to { transform:
rotateY(360deg); } }  @keyframes navSpin3 { from { transform: rotate(45deg);
} to { transform: rotate(405deg); } } </style>
```

8.3 System Status Pill

The status pill reflects the aggregate health of all three cores. Its color, icon, and text update reactively from the Zustand store.

Condition	State Text	Dot Color	Animation
All cores > 80% AND no active scenario	ALL SYSTEMS NOMINAL	#38d97a (green)	None
Any core 40-80% OR active non-critical scenario	DEGRADED STATE	#f59e0b (amber)	Slow pulse (2s)
Any core < 40% OR critical scenario active	CRITICAL ALERT	#ef4444 (red)	Fast flash (0.5s)

Section 9 — Scenarios Full Definition Table

All six drift scenarios are defined below. These definitions must be implemented exactly as specified in `constants/scenarios.ts`. Each row represents one complete scenario.

ID	Title	Severity	Duration	Cores Affected	Visual Effects	Recovery Sequence
<code>narrativeDrift</code>	NARRATIVE DRIFT	MEDIUM	8s	Narrative	Blue ring dims and flickers at reduced intensity. Mirror A fractures (node disappears). Narrative orbital glyphs scramble — random characters replace configured glyphs.	Cognitive ring overpowers (value to 99) → Mirror A heals (node reappears) → Narrative coherence restores to 96+ → System nominal.
<code>cognitiveFailure</code>	COGNITIVE FAILURE	HIGH	10s	Cognitive	Purple ring distorts with high drift wave amplitude	Render ring compensates (fidelity

ID	Title	Severity	Duration	Cores Affected	Visual Effects	Recovery Sequence
					(getDriftRy at max). Scanning grid overlay breaks (grid lines fragment). Red halo glow appears around cognitive ring. AAIS state changes to ALERT.	boosts) → Narrative ring stabilizes (coherence holds at 95+) → Cognitive re-locks (AAIS returns to LOCKED) → Grid overlay restores.
renderCorruption	RENDER CORRUPTION	HIGH	6s	Render	Red ring goes dark (opacity drops to near-zero). Beams scatter outward at random angles. Mirror C fractures. Pipeline state changes to ERROR. Brief canvas noise overlay applied (random dot field over engine area).	Narrative ring re-derives meaning (coherence spikes) → Render ring begins reconstruction (pipeline to BUILDING) → Beams re-converge → Pipeline clears to CLEAR → Mirror C heals.
tripleMirror	TRIPLE MIRROR MISMATCH	CRITICAL	15s	All	All three mirrors fracture simultaneously. All rings dim to 30% glow intensity.	Governance ring locks (speed reduces, containment lines glow bright

ID	Title	Severity	Duration	Cores Affected	Visual Effects	Recovery Sequence
					State orb pulses solid red at 2Hz. Governance ring node labels display ERR fragments. All core values drop to 35-45 range.	green) → Mirror A corrects (3s) → Mirror B corrects (6s) → Mirror C corrects (9s) → Orb pulse returns to normal → All rings restore to nominal values.
orbInstability	STATE ORB INSTABILITY	CRITICAL	12s	All	Orb flashes alternating red and white at 4Hz. All three rings desync — their rotationSpeed values become randomized. Governance ring turns red. All coherence scores drop. Orb coherence Score drops to near zero.	Orb forced re-initialization (orb goes dark for 1.5s) → Orb restores with coherence Score at 50 → Rings re-sync sequentially (Narrative → Cognitive → Render) → Governance ring validates (returns to green) → Full system nominal.
fullRecovery	FULL SYSTEM RECOVERY TEST	TEST	20s	All (sequential)	Stage 1 (0s): Narrative ring fails. Stage 2 (4s): Cognitive	Fully autonomous sequential recovery demonstrating all

ID	Title	Severity	Duration	Cores Affected	Visual Effects	Recovery Sequence
					compensates while Narrative is still degraded. Stage 3 (8s): Render corrupts — triple failure state. Stage 4 (13s): Orb coherence stabilizes all three. Stage 5 (17s): Full synchronized recovery.	four self-healing mechanisms operating together. Ends with all metrics above 95 and event log showing complete recovery chain. Ideal for demo/presentation use.

Section 10 — Responsive Breakpoints

```
/* — Full Desktop Layout (≥ 1000px) ————— */ @media
(min-width: 1000px) { .monitor-layout { grid-template-columns: 280px
1fr 280px; /* 3-column */ } } /* — Tablet / Collapsed Layout (≤ 999px)
————— */ @media (max-width: 999px) { .monitor-layout {
grid-template-columns: 1fr; /* canvas takes full width */ } /* Side
panels slide up from bottom as drawers */ .data-input-panel, .telemetry-
panel { position: fixed; bottom: 0; left: 0; right: 0;
height: 50vh; background: var(--bg-surface); transform:
translateY(100%); /* hidden by default */ transition: transform
var(--transition-med); z-index: 100; overflow-y: auto; padding:
var(--space-lg); border-top: 1px solid var(--border-
active); } .data-input-panel.open, .telemetry-panel.open
{ transform: translateY(0); /* slide up to visible */ } /*
Toggle buttons float above the canvas to open panels */ .panel-toggle-bar {
position: fixed; bottom: 0; left: 0; right: 0; display: flex;
justify-content: space-around; z-index: 101; } } /* — Mobile
Navigation (≤ 640px) ————— */ @media (max-width:
640px) { .tab-label { display: none; } /* icons only in nav
*/ .nav-status-text { display: none; } /* hide verbose status text
*/ .nav-clock { display: none; } /* hide clock to save space */ }
```

Section 11 — Deployment Guide

11.1 Build Commands

```
# Install dependencies npm install # Development server with hot module replacement npm run dev # Production build – outputs to /dist npm run build # Local preview of production build (tests the built artifact) npm run preview
```

11.2 Vercel (Recommended)

```
# Install Vercel CLI (one-time) npm i -g vercel # Deploy to production vercel --prod # Vercel auto-detects Vite. Zero config required. # No environment variables needed.
```

11.3 Netlify

```
# Option A: Connect GitHub repo in Netlify dashboard # Build command: npm run build # Publish directory: dist # Option B: Netlify CLI npm i -g netlify-cli netlify deploy --prod --dir=dist
```

11.4 GitHub Pages

```
// vite.config.ts – set base path to your GitHub repo name import { defineConfig } from 'vite'; import react from '@vitejs/plugin-react'; export default defineConfig({ plugins: [react()], base: '/tri-core-engine/' }); // IMPORTANT: must match your repo name exactly # Build and deploy to gh-pages branch npm run build npm i -g gh-pages gh-pages -d dist
```

11.5 Custom Domain & General Notes

- Any static file host works — no server-side rendering, no API calls, no environment variables.
 - The `dist/` folder is fully self-contained and can be dropped into any CDN or static host.
 - For custom domains: configure your DNS to point to Vercel/Netlify/GitHub Pages per their respective documentation. The app requires no additional server configuration.
 - HTTPS is strongly recommended (Clipboard API requires a secure context for the Share Link copy-to-clipboard feature).
-

Section 12 — Known Limitations & Future Roadmap

Limitation	Current State	Future Solution
Live Feed CORS	User-supplied Live Feed URLs must return CORS headers permitting the app's origin. No-CORS mode is unusable because the response body cannot be read.	Add optional proxy endpoint config field — user supplies their own CORS proxy URL. Or provide a minimal server-side relay worker (Cloudflare Workers / Vercel Edge Function).
PNG Export Scale Limits	Limited by browser canvas maximum size (~16,384px per dimension on most browsers). At 2× scale on a large monitor this may silently produce a blank export in some browsers.	Use <code>OffscreenCanvas</code> with a Web Worker for off-thread rendering, or add a server-side headless-browser render endpoint (Puppeteer/Playwright).
No Auth / Accounts	Config lives exclusively in <code>localStorage</code> . Clearing browser data loses all saved configuration. No cross-device sync.	Add Supabase auth + cloud config storage. Config syncs to user account. Share links become short persistent URLs.
Share Link Size	Config encoded as base64 in URL hash. Safe limit is approximately 2KB of JSON (~1,300 characters of config). Very large custom configs may exceed URL length limits in some environments.	Backend shortlink service (generate a short token that maps to a stored config blob). Eliminates URL size concern entirely.
Web Workers	The entire animation loop including particle system runs on the main thread. On very low-end hardware, heavy particle counts may compete with React rendering.	Move the particle system to an <code>OffscreenCanvas</code> + Web Worker. Main thread handles only React UI; worker owns the canvas.
No telemetry persistence	Telemetry history (sparkline data) is entirely in-memory and lost on page refresh.	Store last N readings in IndexedDB for session continuity and longer-term trend analysis.

Appendix A — Glyph Set

The following characters constitute the full glyph set available for selection on the Narrative Ring orbital nodes. Defined in `constants/glyphs.ts`. The default selection is: $\Omega \Sigma \infty \Delta \Psi$.

```
// constants/glyphs.ts export const AVAILABLE_GLYPHS: string[] = [ 'Ω',  
'Σ', '∞', '<', '>', '§', '℥', 'Δ', 'Λ', 'Ψ', 'Φ', 'Θ', 'ξ', 'π', 'μ', 'ν',  
'∂', '∇', '⊕', '⊗', '⋄', '⊙', '▲' ]; export const DEFAULT_GLYPHS: string[] =  
[ 'Ω', 'Σ', '∞', 'Δ', 'Ψ' ];
```

Glyphs are drawn as text using the mono font at the orbital node positions on the Narrative ring. Font size scales with `minDim` — approximately `Math.max(10, minDim * 0.022)px`. Use `ctx.textAlign = 'center'` and `ctx.textBaseline = 'middle'` when drawing.

Appendix B — Color Reference Table

Name	Hex / Value	CSS Token	Usage
Void Background	#020810	--bg-void	Page background — outermost layer
Surface	#080f1e	--bg-surface	Cards, side panels, telemetry containers
Elevated	#0d1728	--bg-elevated	Dropdowns, modals, tooltips
Narrative Blue	#3b9eff	--color-narrative	Ring 1 stroke, sparklines, CoreStatusCard accent, log badge
Cognitive Purple	#b06ef5	--color-cognitive	Ring 2 stroke, sparklines, CoreStatusCard accent, log badge
Render Red	#f06060	--color-render	Ring 3 stroke, sparklines, CoreStatusCard accent, log badge
Governance Green	#38d97a	--color-governance	Outer ring, status

Name	Hex / Value	CSS Token	Usage
			NOMINAL dot, log ok entries
Warning Amber	#f59e0b	--color-warn	DEGRADED state pill, warn log entries, DRIFTING badge
Error Red	#ef4444	--color-error	CRITICAL pill, error log entries, ALERT badge, orb instability
Text Primary	#e8edf5	--text-primary	All primary readable body text
Text Muted	#5a6a82	--text-muted	Secondary labels, timestamps, info log entries
Text Dim	#2a3a52	--text-dim	Placeholder text, disabled UI elements
Border Subtle	rgba(100,160,255,0.12)	--border-subtle	Default card and panel borders
Border Active	rgba(100,160,255,0.30)	--border-active	Focused inputs, hovered cards, active controls

Appendix C — Quick-Start Checklist for the Developer

Use this checklist to track implementation progress from project initialization to production deployment. Items are ordered to minimize blocked dependencies.

5. ☐ **Init repo** — `npm create vite@latest tri-core-engine -- --template react-ts`.
Verify dev server runs.
6. ☐ **Install Zustand** — `npm i zustand`. Confirm TypeScript types resolve.
7. ☐ **Set up CSS tokens** — Copy all custom properties from Section 7 into `src/index.css`. Add global reset and scrollbar styles.
8. ☐ **Build EngineStore** — Implement all four slices (config, telemetry, log, scenario) + persist middleware. Validate with a test log push. (Section 4)

9. ☐ **Set up EngineCanvas** — Implement the RAF loop hook, canvas DPR scaling, and ResizeObserver. Confirm a test rect draws centered and scales on window resize. (Section 5.2)
10. ☐ **Implement renderer.ts** — Add all 7 draw layer stubs, then flesh out each: particles → governance → mirrors → narrative → cognitive → render → orb. Verify draw order produces correct Z-order. (Section 5)
11. ☐ **Build NavBar** — Implement SVG logo, 5 tab buttons, status pill logic, live clock. Wire tab switching to route state in App.tsx. (Section 8)
12. ☐ **Build Monitor tab** — DataInputPanel (3 modes) + EngineCanvas center column + TelemetryPanel (3 CoreStatusCards + StateOrbCard + EventLog). Verify simulated data updates canvas visually. (Section 6.1)
13. ☐ **Build Configure tab** — RingConfigurator accordions for all 4 rings + OrbConfigurator + ThemePresets (4 themes). Verify every control reflects immediately on the Monitor canvas. (Section 6.2)
14. ☐ **Define all 6 scenarios** — Implement constants/scenarios.ts with all ScenarioDef objects per Section 9. Run a console log validation that all phase timestamps are sequential.
15. ☐ **Implement ScenarioRunner** — Build the singleton class with start/cancel methods and full setTimeout timeline. Test with narrativeDrift scenario. (Section 6.3)
16. ☐ **Build Simulate tab UI** — ScenarioCard grid (6 cards) + RecoveryTimeline progress bar + active phase label + cancel button. Wire to ScenarioRunner singleton.
17. ☐ **Build Export tab** — PNG download (3 scale options) + JSON config download + ShareLink component with clipboard copy + CSV telemetry export. (Section 6.4)
18. ☐ **Build Docs tab** — Static JSX sections with sidebar anchor navigation. Use React.lazy() for each section. (Section 6.5)

19. ☐ **Implement URL hash loading** — Add `useEffect` in `App.tsx` to call `loadFromHash()` on mount and apply config if present. Test full round-trip: generate link → open in new tab → verify config restored. (Section 6.4)
20. ☐ **Add responsive breakpoints** — Implement tablet drawer behavior and mobile nav icon-only mode. Test at 768px and 375px viewport widths. (Section 10)
21. ☐ **Integration test checklist:**
- Config persistence: change a ring color → refresh page → color preserved ✓
 - Share link round-trip: generate link → open incognito → config loaded ✓
 - All 6 scenarios: run each to completion, verify log entries match spec ✓
 - PNG download: export at all 3 scales, verify file downloads and renders ✓
 - Manual JSON: paste valid payload → inject → canvas responds ✓
 - Theme presets: apply all 4 themes → colors update immediately ✓
22. ☐ **Build and deploy** — `npm run build` → verify dist output → deploy to Vercel or Netlify → confirm production URL loads correctly. (Section 11)

Estimated development time: 40–60 hours solo developer | 20–30 hours with 2 developers (recommended split: one owns canvas/engine, one owns UI modules and state).